

CISP 400

Object Oriented Programming in C++

Professor Fowler



F O L S O M L A K E C O L L E G E
EL DORADO CENTER | RANCHO CORDOVA CENTER

Last Time

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data Types

8.13 Arrays of Objects

Theme for Today

“Method.”

Chapter 7

Object Oriented Programming

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

**7.2 Object-Oriented
Programming**

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

Code Together

Programming Exercise

Exercise Goals

- Increase Topic Mastery
- Construct Knowledge
- Give you data to evaluate how well you are learning.

Programming Exercise

- One issue with functions in C++ is the limitation of moving only 1 variable out with a return statement.
- Create a struct with string student and int grade. Pass both of these elements out of the function Foo() into main(). Try to avoid creating global variables with your struct.

```
1 //pe070101.cpp -- Class Exercise
2 #include <iostream>
3 #include <string>
4 using namespace std;
5
6 struct Student {
7     string name;
8     int grade;
9 };
10
11 Student Foo();
12
13 int main() {
14     Student s;
15     s = Foo();
16     cout << s.name << ": " << s.grade << endl;
17 }
18
19 Student Foo() {
20     Student tmp;
21     tmp.name = "Pat Smith";
22     tmp.grade = 95;
23     return tmp;
24 }
```

7.3 Introduction to Classes

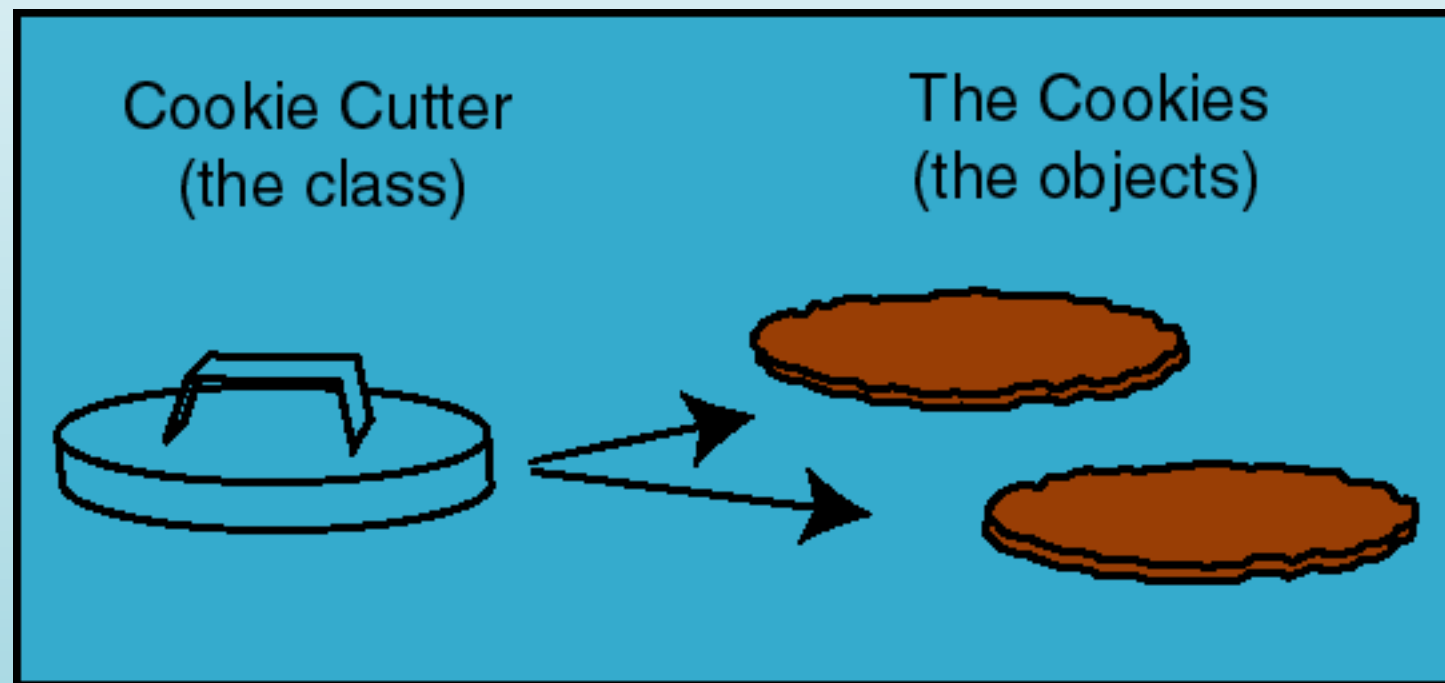


Photo Credit: <https://qph.ec.quoracdn.net/main-qimg-021242ab769010da96e2a2ecb7fb266a>

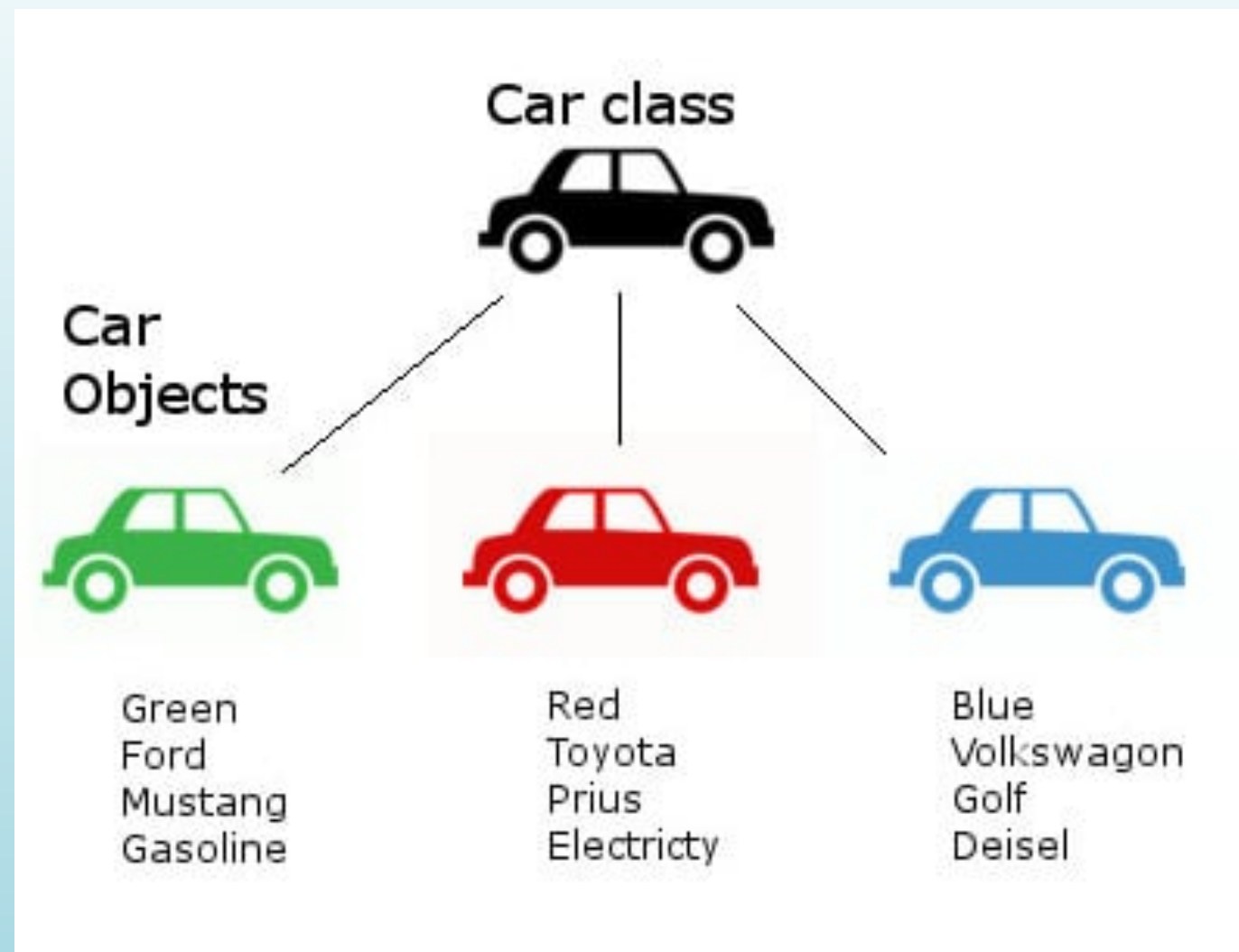


Photo Credit: https://3.bp.blogspot.com/-y8do_JdFx6c/V3_dywgZzPI/AAAAAAAAAGZY/catSBAX1g-ciuT6LxvvnjYAfWEWH2jAmQCLcB/s1600/Wordane-OOP-car-class-eg-2.jpg

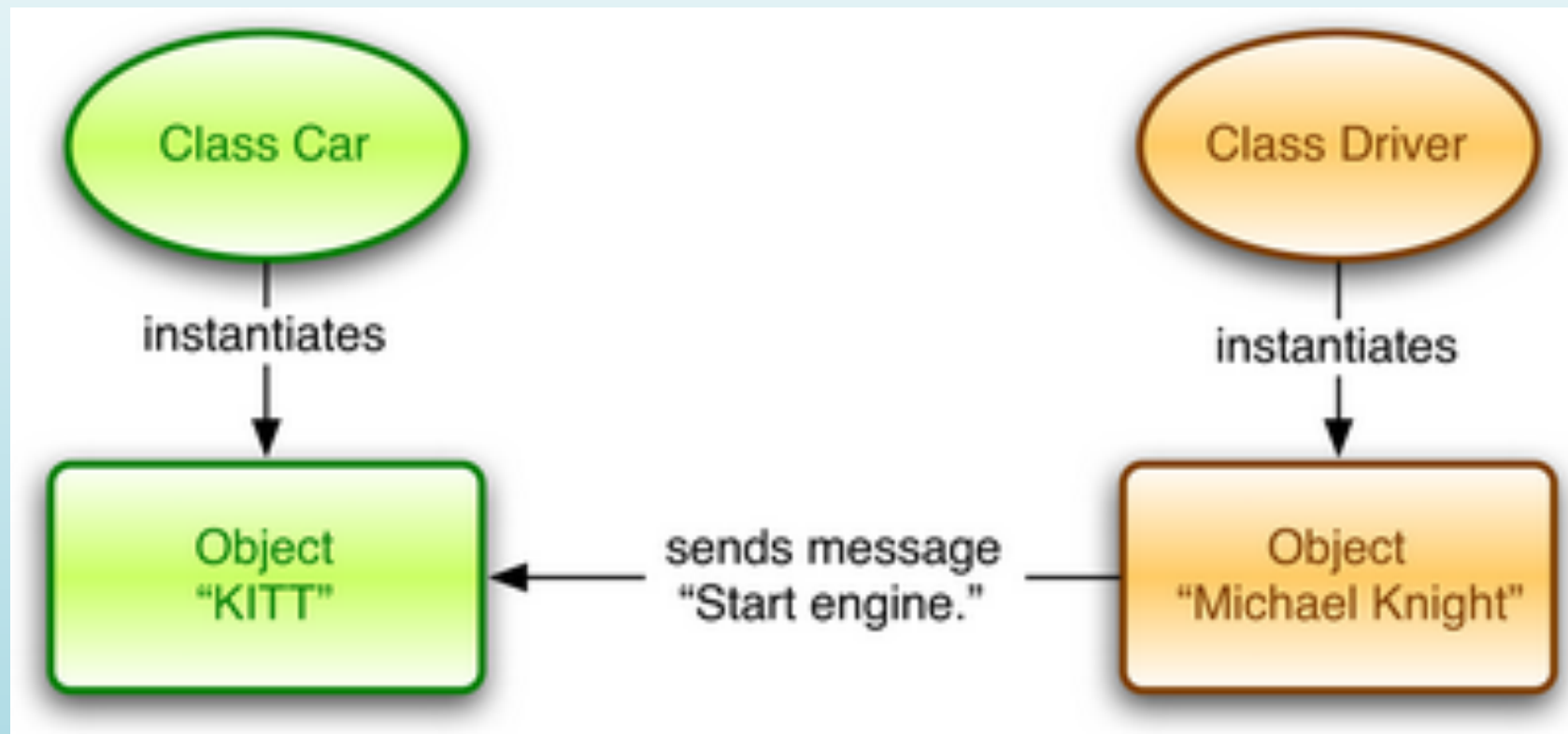


Photo Credit: <https://iandunn.name/content/presentations/wp-oop-mvc/images/object-instantiation.png>

7.4 Creating and Using Objects

```
// pointClass.cpp -- Class demo
#include <iostream>
using namespace std;

int main() {
    int x, y;
    x = 15;
    y = 33;

    cout << "Object is located at " <<
        x << ", " << y << endl;
}
```

```
// pointClass1.cpp -- Class demo
#include <iostream>
using namespace std;

class Point {
private:
    int x = 15;
    int y = 33;
public:
    void Print() {
        cout << "Object is located at " <<
            x << ", " << y << endl;
    }
};

int main() {
    Point widget1;

    widget1.Print();
}
```

7.5 Defining Member Functions

- Classes group data and associated behaviors.
- How do we create those behaviors?

- We use methods!

Methods

FORCED functions :. Forced abstraction.

Everything you learned about functions in CISP 360 apply.

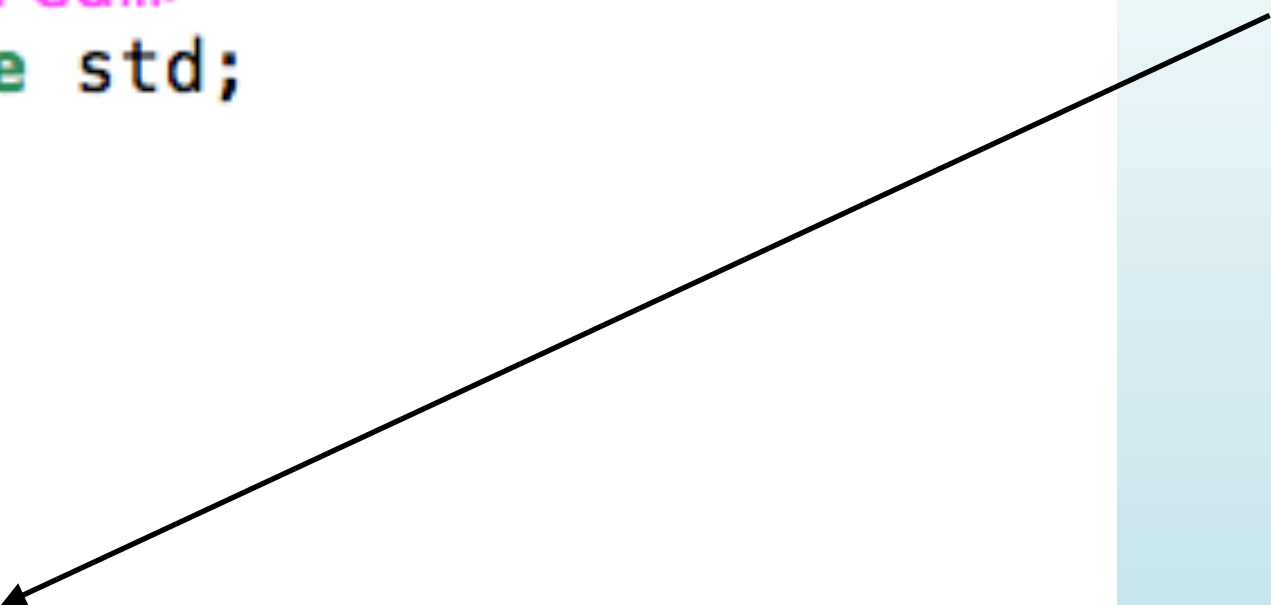
```
// pointClass1.cpp -- Class demo
#include <iostream>
using namespace std;

class Point {
private:
    int x = 15;
    int y = 33;
public:
    void Print() {
        cout << "Object is located at " <<
            x << ", " << y << endl;
    }
};

int main() {
    Point widget1;

    widget1.Print();
}
```

Method



Another example


```

1 // stoplight1.cpp
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 class StopLight
8 {
9     private:
10         bool isPowered = false;
11         int circuitBreaker = 3;
12
13     public:
14         void PowerOn();
15         void PowerOff();
16         void Cycle();
17 };
18 void StopLight::PowerOn()
19 {
20     isPowered = true;
21 }
22 void StopLight::PowerOff()
23 {
24     isPowered = false;
25 }

```

```

26 void StopLight::Cycle()
27 {
28     int i = 0;
29     while (isPowered)
30     {
31         cout << "Green" << endl;
32         for (int i = 0; i < 100000000; i++)
33             { ; }
34         cout << "Yellow" << endl;
35         for (int i = 0; i < 100000000; i++)
36             { ; }
37         cout << "Red" << endl;
38         for (int i = 0; i < 100000000; i++)
39             ;
40         if (i < circuitBreaker)
41             i++;
42         else
43             PowerOff();
44     }
45 }
46
47 int main()
48 {
49     StopLight bidwell;
50     bidwell.PowerOn();
51     bidwell.Cycle();
52 }

```

7.6 Constructors

Constructors

- Run when the class is instantiated.
- Same name as the class.
- No return type at all (not even void).

- Does this mean classes which we have not created a constructor, do not have constructors?

Constructor Use Case

Set up connections between objects (DB, network, etc)

Initialize data.

Serve as an interface.

Constructors as an interface

We usually have data we want to initialize with when we instantiate our constructor.

What if we don't have all the data? Does that mean we can't instantiate?

Overloading Constructors

- Computer always makes a default constructor - unless you write ANY other constructor.
- Meaning: you will not have a default constructor if you've overloaded it.
- Meaning: you cannot create an object when you do not pass any values upon instantiation. Refuses to compile.
- Typical default constructor is not passed any arguments.
- However, we can have a default constructor with arguments if we set the parameters with default values.
- We can only have one default constructor. We can have unlimited number of overloaded constructors.

Bottom Line

- If you are adding a constructor which accepts arguments, you will also need to create a default constructor.
- A default constructor does not accept any arguments, unless you make default parameters available.

```

1 // 7-6.cpp -- Overloading constructors
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 class Sale {
7 private:
8     double taxRate;
9 public:
10    Sale(double rate)           // constructor with 1 parameter
11    {    taxRate = rate;
12    }
13
14    Sale()                     // default constructor
15    {    taxRate = 0.0;
16    }
17
18    double calcSaleTotal(double cost) {
19        double total = cost + cost * taxRate;
20        return total;
21    }
22 };
23
24 int main()
25 {
26     Sale cashier1(.06);
27     Sale cashier2;
28
29     // format the output
30     cout << fixed << showpoint << setprecision(2);
31
32     // Get and display the total sale price for two $24.95 sales
33     cout << "With a 0.06 sale tax rate, the total \n";
34     cout << "of the $24.95 sale is $";
35     cout << cashier1.calcSaleTotal(24.95) << endl;
36
37     cout << "\nOn a tax-exempt sale, the total\n";
38     cout << "of the $24.95 sale is $";
39     cout << cashier2.calcSaleTotal(24.95) << endl;
40     return 0;
41 }

```

Constructors, functions
and methods, Oh my!

- Distinguishing between various constructors and methods can be confusing.

```
Class ABC {  
    };
```

```
int main() {  
    ABC x;          // Default - works fine  
    ABC x();        // Error - thinks it's a method  
    ABC x(10)       // Fine - overloaded default
```

7.7 Destructors

```

1 // stoplight2.cpp
2 #include <iostream>
3 #include <string>
4 using namespace std;
5
6 class StopLight {
7     private:
8         bool isPowered = false;
9         int circuitBreaker = 3;
10        int delay = 0;
11
12    public:
13        StopLight();
14        ~StopLight();
15        void PowerOn();
16        void PowerOff();
17        void Cycle();
18 };
19 StopLight::StopLight() {
20     cout << "Powering on ..." << endl;
21     delay = 100000000;
22 }
23 void StopLight::PowerOn() {
24     isPowered = true;
25 }
26 void StopLight::PowerOff() {
27     isPowered = false;

```

```

28 }
29 void StopLight::Cycle() {
30     int i = 0;
31     while (isPowered)
32     {
33         cout << "Green" << endl;
34         for (int i = 0; i < delay; i++)
35             { ; }
36         cout << "Yellow" << endl;
37         for (int i = 0; i < delay; i++)
38             { ; }
39         cout << "Red" << endl;
40         for (int i = 0; i < delay; i++)
41             ;
42         if (i < circuitBreaker)
43             i++;
44         else
45             PowerOff();
46     }
47 }
48 StopLight::~~StopLight() {
49     cout << "Powering Off ..." << endl;
50 }
51
52 int main() {
53     StopLight bidwell;
54     bidwell.PowerOn();
55     bidwell.Cycle();
56 }

```




- Constructors!
- Destructors!

Coding Exercise

Create a Date class with private member ints day, month, and year. Add a setDate() method to set the date. Add a print public member function to display the date.

Write a test rig to exercise this class.



What's this section
about?

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

**7.2 Object-Oriented
Programming**

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

What's Next?

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

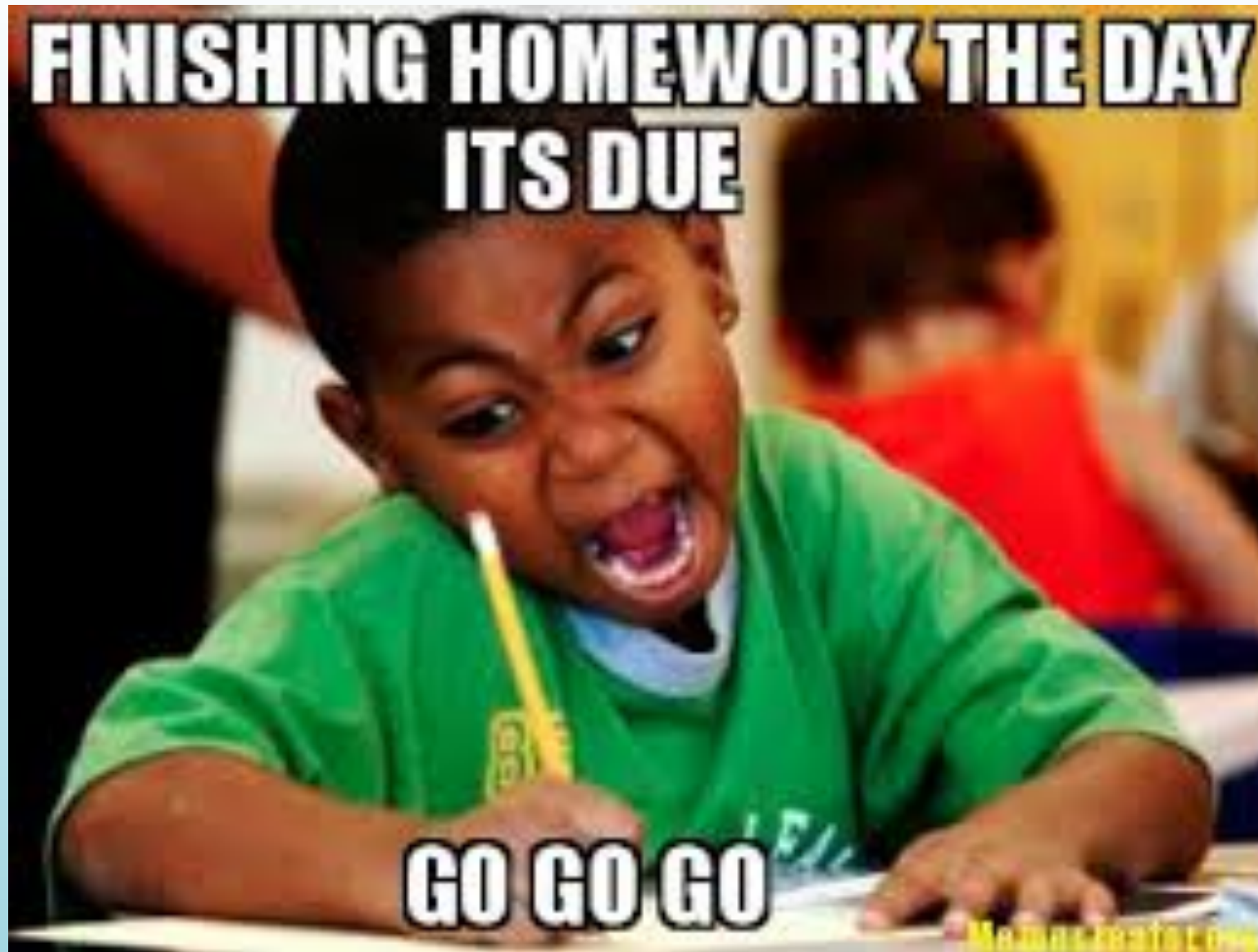
7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects



Homework 2 due Sunday.

Modify This


```

1 // stoplight1.cpp
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 class StopLight
8 {
9     private:
10         bool isPowered = false;
11         int circuitBreaker = 3;
12
13     public:
14         void PowerOn();
15         void PowerOff();
16         void Cycle();
17 };
18 void StopLight::PowerOn()
19 {
20     isPowered = true;
21 }
22 void StopLight::PowerOff()
23 {
24     isPowered = false;
25 }

```

```

26 void StopLight::Cycle()
27 {
28     int i = 0;
29     while (isPowered)
30     {
31         cout << "Green" << endl;
32         for (int i = 0; i < 100000000; i++)
33             { ; }
34         cout << "Yellow" << endl;
35         for (int i = 0; i < 100000000; i++)
36             { ; }
37         cout << "Red" << endl;
38         for (int i = 0; i < 100000000; i++)
39             ;
40         if (i < circuitBreaker)
41             i++;
42         else
43             PowerOff();
44     }
45 }
46
47 int main()
48 {
49     StopLight bidwell;
50     bidwell.PowerOn();
51     bidwell.Cycle();
52 }

```